

Task 13: Synchronizing with Effects

Summary

In this task, I learned about Synchronizing with Effects using the `useEffect()` Hook in React. I understood that Effects are used to perform side effects after a component renders, such as fetching data from APIs, updating the document title, setting timers, subscribing to events, or interacting with external libraries. Unlike event handlers, Effects run automatically after rendering to synchronize the UI with external systems.

I also learned how the dependency array controls when an Effect executes. An empty dependency array (`[]`) makes the Effect run only once after the component is mounted, while specifying dependencies causes it to run only when those values change. This helps improve performance by avoiding unnecessary executions and ensures that the component stays synchronized with the latest data.

Another important concept I learned was the cleanup function in `useEffect()`. Cleanup functions are used to remove event listeners, clear timers, or cancel subscriptions before the component unmounts or before the Effect runs again. This prevents memory leaks and keeps React applications efficient and well-organized.

Learning Outcome

After completing this task, I understood how to use `useEffect()` to handle side effects, control its execution with dependency arrays, and clean up resources properly. These concepts are essential for building efficient, responsive, and maintainable React applications.

Task 14: Build a User List from API

Summary

In this task, I learned how to build a User List application by fetching user data from the JSONPlaceholder API using React. I used the `useState()` Hook to store the fetched data and the `useEffect()` Hook to make the API request when the component was first rendered. This allowed me to display real-time data instead of using static content.

I also learned how to make asynchronous API requests using the `fetch()` method, convert the response into JSON format, and update the component state with the received data. Once the data was stored, I used the `map()` method to render each user's information dynamically, making the application scalable and reusable.

Additionally, I understood the importance of handling different application states such as loading, successful data retrieval, and errors. Managing these states improves the user experience and makes the application more reliable. This task gave me practical experience in integrating external APIs into React applications.

Learning Outcome

After completing this task, I understood how to fetch data from an external API, manage it using `useState()` and `useEffect()`, render dynamic lists using `map()`, and handle loading and error states effectively. These concepts are essential for developing real-world, data-driven React applications.

⇒ **GITHUB REPO LINK:**

https://github.com/yashrathod910/15D_Internship/tree/main/DAY7